Cloud-Based ML/AI System Design submission

Bryan Cortes, Earl Padron, Irwin Salamanca, Matthew Tang

1. Problem Statement & AI Objective

Real-World Problem
Modern IoT and edge applications are increasingly relying on cloud serverless platforms to process data generated by distributed devices. While cloud execution provides massive scalability and centralized management, it introduces unavoidable network latency, bandwidth overhead, and performance variability caused by container "cold starts". Conversely, edge devices running frameworks like AWS IoT Greengrass can process data locally with significantly lower latency and improved resilience, but they are bottlenecked by limited compute resources.

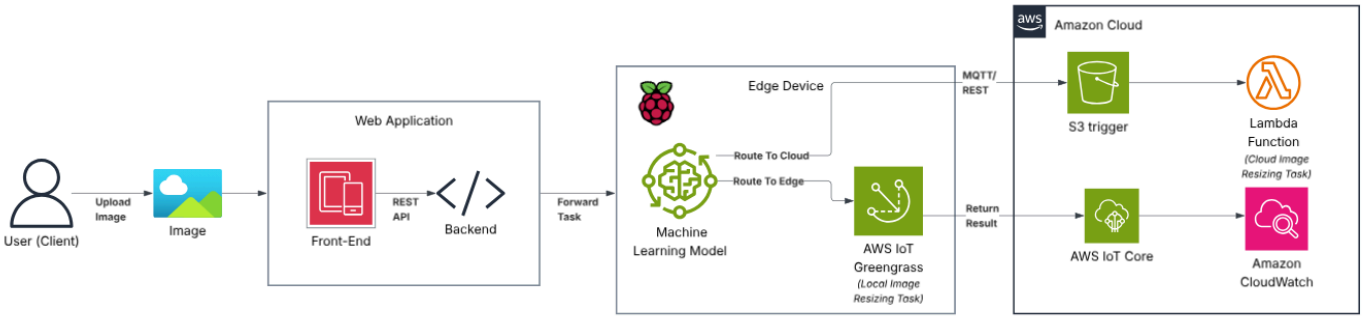
Hardcoded systems where it is either strictly only edge computing or cloud computing fail to adapt to changing workload sizes, fluctuating network conditions, and device resource constraints.

Target Users
The target users for this system are end-users that need a seamless web interface for responsive image processing, as well as cloud engineers building latency-sensitive hybrid edge applications

AI Task
The system's core intelligence relies on Regression Modeling and Optimization Algorithms. For each incoming task (e.g., an image resizing request), the AI component extracts task features (e.g., input size, edge CPU utilization, network RTT) to predict three specific variables: edge execution latency, cloud execution latency, and cloud execution cost. A decision algorithm then applies a placement policy to select the optimal execution location.

Learning Paradigm: This relies on Supervised Learning. The data-driven latency and cost prediction models are trained from historical metrics gathered during measured benchmark experiments.

2. High-Level Cloud Architecture Diagram



3. ML Pipeline Design

A. Data Collection

Since execution latency and cost vary based on various conditions specific to each deployment (hardware of the device, network conditions, AWS configuration), we will create our own custom supervised dataset based on the results of the benchmarking experiment. For each image resize request, the same workload will be repeated for both environments.

Each experiment generates one dataset record containing:

- **Input features (*X*)**: Image size(bytes), file format, edge CPU utilization (%), edge memory utilization (%), network round-trip time (RTT in ms), estimated uplink bandwidth (kbps), Lambda memory allocation (MB), and cold start indicator(0 or 1).
- **Target Variables (*y*)**: Edge total latency (ms), Cloud total latency (ms), and estimated cloud cost (USD)

The logs will be collected from various sources: AWS CloudWatch, system logs from the Raspberry Pi, and custom logs for the timing, which are stored in Amazon S3.

B. Data Preprocessing

The preprocessing state transforms raw logs into a clean training dataset.

- **Log Parsing and Aggregation:** Execution logs are parsed to obtain timing information and pair edge and cloud executions based on request IDs.
- **Feature Engineering:** includes log transformations of image size, encoding categorical variables (file format), normalization of continuous features, and aggregation of recent network statistics (e.g, rolling RTT average).
- **Outlier Handling:** Removal or trimming of extreme latency outliers and filtering out corrupter records.
- **Train/Validation/Test Split:** to simulate real-world deployment and prevent temporal leakage, data is split chronologically(earlier data for training, recent data for validation/test).

C. Model Training

The model training takes place on cloud edge compute resources (such as an EC2 instance or an Amazon SageMaker notebook) or a local developer machine. The edge device (Raspberry Pi) is exclusively used for running real-time inference via the serialized models. The framework utilizes supervised regression algorithms:

- **Edge Compute Prediction:** Uses Ridge Regression to serve as a fast, mathematically lightweight baseline that captures the linear scaling of local edge image processing.
- **Cloud Compute Prediction:** Uses Gradient Boosted Regression Trees(GBRT) to accurately capture the non-linear relationship between image size, network latency fluctuations, and specific Lambda memory allocations.

Models are evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the 95th percentile absolute error (P95), and compared against a rule-based baseline to prove data-driven efficacy.

D. Model Deployment & Real-Time Policy Routing

The trained model is then serialized (joblib/ONNX) and deployed to the Raspberry Pi as part of a AWS Greengrass component via over-the-air updates. Most importantly, the final routing is determined by a Decision Engine Policy via a local REST API endpoint:

- **Concurrency Hard Limit(User Activity Checking):** This endpoint checks the active queue of concurrent tasks. If the active user activity exceeds the maximum allowed concurrent threshold for the Raspberry Pi, the policy bypasses the ML model altogether and routes to the cloud.
- **Budget & Latency Optimization:** if the edge is not busy, the ML model predictions are evaluated. The Python code will route to the cloud if `predicated_cloud_latency < predicted_edge_latency AND predicted_cloud_cost <= user_budget`; otherwise, it will route to the edge.

4. Cloud Component Design

Web Application (Frontend & Backend)

- **Role:** provides a UI for users to upload images and view resized outputs. The backend tracks the active user sessions to determine system concurrency.
- **Technology & Justification:** Python with FastAPI framework. FastAPI provides high-performance asynchronous I/O to handle concurrent image uploads without blocking.
- **Communication**: Exposes RESTful endpoints to the frontend and forwards HTTP payload to the edge node (Raspberry Pi).

AWS IoT Greengrass (Edge Runtime)

- **Role:** Acts as the execution environment on the physical Raspberry Pi hardware.
- **Justification:** Allows the execution of serverless AWS Lambda functions locally on edge devices while seamlessly syncing state, data, and ML model updates via Over-the-Air (OTA) with the broader AWS ecosystem.
- **Communication:** Interacts locally with the data source and communicates with AWS IoT Core via the MQTT protocol over TLS.

AWS Lambda (Cloud Compute)

- **Role:** Handles the heavy-lifting cloud execution path when the Decision Engine routes a task away from the edge.
- **Justification:** A fully managed Function-as-a-Service (FaaS) platform that scales automatically. It integrates with S3 event triggers.
- **Communication:** Triggered by object creation events in S3; writes computed results back to an S3 bucket.

Amazon S3 (Data Storage)

- **Role:** Serves as the central repository for raw data payloads routed to the cloud, as well as the final destination for processed image outputs.
- **Justification:** Provides 99.999999999% durability, massive scalability, and cheap object storage. It is the industry standard for unstructured data like images.
- **Communication:** Acts as the storage layer for both the AWS IoT Core rules engine and the AWS Lambda outputs.

Custom Decision Engine

- **Role:** The "brain" of the operation running at the edge. It applies the placement policy: either minimizing latency under a cost constraint or minimizing cost under a latency deadline.
- **Justification:** By placing the decision logic at the edge, the system avoids the

network round-trip penalty of asking the cloud where to execute the task

5. Scalability & Performance Strategy

Auto-Scaling & Load Balancing:
When tasks are routed to the cloud, AWS Lambda inherently provides auto-scaling. It automatically provisions new execution containers (handling cold starts) to match incoming request throughput. Load balancing across these containers is entirely abstracted and managed by the AWS infrastructure.

Horizontal vs. Vertical Scaling:

- **Cloud:** Scales horizontally by spinning up thousands of concurrent Lambda instances.
- **Edge:** Constrained physically (vertical scaling is limited by the Pi's hardware). To scale horizontally, the Greengrass group configuration can be deployed across a fleet of thousands of Raspberry Pis, turning a single node into a distributed edge cluster.

Latency Optimization & Bottlenecks:
The main bottleneck in the system is the edge CPU degradation due to concurrent user activity. The suggested framework addresses this latency bottleneck through the policy that limits concurrent users. Through monitoring workloads and reactively redirecting overflow traffic to an elastic cloud environment within AWS, the system can guarantee high responsiveness even during massive traffic spikes.

Proactive Cold Start Mitigation (Asynchronous Warming Pings):

While the ML models are able to actively predict and avoid AWS Lambda “Cold Starts”, the suggested framework can go a step further by implementing a proactive mitigation strategy to optimize the cloud path for concurrent traffic. In cases when the system detects that the cloud is “cold” and routes a user’s image processing request to the Raspberry Pi for immediate processing, a Python FastAPI backend script can simultaneously fire a lightweight, asynchronous “Warming Ping” (an empty payload) to the AWS Lambda payload to prime for the next user request. This background request forces AWS to initialize the container and load the necessary Python image processing libraries without blocking the active user’s edge request that is executing at the same time. Consequently, when the concurrent user uploads an image moments later, the path to the cloud is fully “warm” and is promptly available to provide sub-second latency, ensuring the system can leverage AWS lambda’s horizontal elasticity.

6. Security & Governance

Authentication & Authorization: The Raspberry Pi utilizes X.509 certificates over mutual TLS. The Raspberry Pi certificate is attached to a dedicated AWS IoT policy, which restricts MQTT publish and subscribe permissions to specific authorized topics only. For AWS service access, the Greengrass core device uses the Token Exchange Service (TES) with an IAM role alias. This IAM role follows the Principle of Least Privilege.

IAM roles and permissions: Below are the IAM roles we will be utilizing:

- **Greengrass Core Device Role**
  - Permission to upload objects only to specific S3 buckets
  - Permission to write logs and custom metrics to CloudWatch
  - No broad administrative privileges
- **Cloud Lambda Execution Role**
  - Permission to read/write to designated S3 buckets
  - Permission to publish monitoring metrics
  - No cross-service or account-wide access

Data Encryption:

- **In Transit:** All communications between the edge and the cloud (whether MQTT payloads to IoT Core or HTTPS uploads to S3) are encrypted using TLS 1.2+.
- **At Rest:** Data stored in Amazon S3 is protected using AES-256 server-side encryption (SSE-S3).

Logging and Monitoring: Amazon CloudWatch serves as the centralized logging service. Both the edge Lambda functions (via Greengrass) and the cloud Lambda functions push execution metrics, error traces, and runtime durations to CloudWatch. This data is critical for training the latency prediction models and monitoring system health.

Backup and disaster recovery strategy: To ensure resilience and data durability:

- Amazon S3 provides 99.9999999% durability, protecting stored image data against loss
- S3 Versioning can be enabled to recover from accidental deletion or overwrites
- Lifecycle policies ensure controlled retention and archival of data
- CloudWatch logs can be retained and exported for audit compliance

In the event of an edge device failure:

- The Raspberry Pi can be reprovisioned using stored IoT certificates and configuration backups
- All cloud-side resources (Lambda functions, S3 buckets, IoT configurations) can be redeployed using Infrastructure-as-Code templates if necessary.

This approach ensures both operational continuity and recoverability.

7. Cost Optimization Strategy

Free Tier Usage: This project will be utilizing free tier resources but will require a paid account. During the 3-month development cycle, the team will leverage the AWS Free Tier which includes 1 million free AWS Lambda requests per month and 5GB of standard S3 storage, keeping prototype operational costs near zero. The reason for using a paid account is to be able to get access to AWS Greengrass which allows for the use of IoT devices. We will be paying for registration (~$0.16 per device) and messages being sent to the IoT core ($1.00 per 1 million messages for the first 1 billion messages).

Serverless vs. Virtual Machines: This system is built entirely on serverless execution (AWS Lambda and Greengrass) rather than virtual machines. The system incurs zero charges for idle compute time.

Spot Instances: This does not apply to our project as the system uses event-driven serverless functions instead of virtual machines.

Storage Lifecycle Policies: To prevent S3 storage costs from ballooning over time due to high-throughput image processing, an S3 Lifecycle Policy will be implemented. Raw images temporarily uploaded to trigger cloud Lambdas will be automatically deleted after 24 hours, and processed thumbnails will be transitioned to cheaper Glacier storage if archiving is required.

Rough Cost Estimation Approach: Cost estimation is calculated using the AWS Pricing Calculator and is based on the following:

- Number of images processed per month
- Average Lambda Execution duration
- Allocated memory configuration
- IoT Core message volume
- Total S3 storage usage

Because the architecture is serverless and event-driven, we expect total costs to scale linearly with workload, allowing for predictable and controllable spending.

Below is a rough cost estimate table depending on how many images enter the system.

Image count	Compute & Request Lambda & S3	Storage of resized images	Outbound Data	Total Monthly Cost
10,000	$0.28	$0.044	$0.17	$0.494

100,00	$2.70	$0.44	$1.72	$4.86
1,000,000	$27.00	$4.39	$17.17	$48.56

While cost ramps up once a very large amount of images are used, the amount of images entering the system will be limited to ensure cost does not increase from abnormal or normal high traffic. Also the only time a very high amount of images will be used is for training purposes which will not incur all the cost associated with outbound data from aws and storage since Sagemaker will be used to train. Offloading cost of training from our system.

8. References

https://arxiv.org/abs/2003.01310
[1] A. Das, S. Imai, M. P. Wittie, and S. Patterson, “Performance Optimization for Edge-Cloud Serverless Platforms via Dynamic Task Placement,” *arXiv preprint arXiv:2003.01310*, 2020.

https://arxiv.org/abs/2307.06397
[2] R. Jin, Q. Yang, and M. Zhao, “When Edge Meets FaaS: Opportunities and Challenges,” *arXiv preprint arXiv:2307.06397*, 2023.

https://arxiv.org/pdf/1811.05948
[3] Anirban Das, Stacy Patterson, Mike P. Wittie, “EdgeBench: Benchmarking Edge Computing Platforms”

https://ieeexplore.ieee.org/document/7807196
[4] M. Satyanarayanan, “The Emergence of Edge Computing”

https://ieeexplore.ieee.org/document/8016573
[5] Y. Mao, C. You, J. Zhang, K. Huang, K. B. Letaief, “A Survey on Mobile Edge Computing: The Communication Perspective”

[6] Amazon Web Services, “Amazon S3 Documentation,” AWS Documentation. [Online]. Available: https://docs.aws.amazon.com/s3/

[7] Amazon Web Services, “AWS IoT Greengrass Documentation,” AWS Documentation. [Online]. Available: https://docs.aws.amazon.com/greengrass/

[8] Amazon Web Services, “AWS Lambda Documentation,” AWS Documentation. [Online]. Available: https://docs.aws.amazon.com/lambda/

[9] Amazon Web Services, “Amazon CloudWatch Documentation,” AWS Documentation. [Online]. Available: https://docs.aws.amazon.com/cloudwatch/